
EMBARCADERO CONFERENCE 2026
Delphi Aumentado por IA
IA
Modernização inteligente de sistemas legados
sem perder o conhecimento do negócio
Prof. Marcelo Nogueira
Delphi
CTA
TECNOLOGIA EDUCACIONAL

MATERIAL DE APOIO DA PALESTRA

Delphi Aumentado por IA

Modernização Inteligente de Sistemas Legados sem Perder o Conhecimento do Negócio

Prof. Marcelo Nogueira | CTA Tecnologia Educacional

Embarcadero Conference 2026 | 01/09/2026

Acesse os materiais: https://bit.ly/econ_mn_2026

Ideia central

Este e-book acompanha a palestra e organiza, em formato prático, a tese de modernização incremental: compreender o legado, explicitar regras de negócio, separar responsabilidades, criar serviços e abrir caminho para web, APIs, nuvem e IA aplicada com responsabilidade.

Sumário

1. Por que esta conversa importa
2. O que realmente envelhece em um sistema
3. A armadilha da reescrita total
4. A tese do Delphi Aumentado por IA
5. Onde a IA ajuda de verdade
6. Modernização inteligente: do medo ao método
7. Demo visual: o caso do Pedido legado
8. Onde entra o TMS WEB Core
9. Arquitetura recomendada com TMS
10. O que muda quando o alvo é web
11. Roteiro prático para modernizar sem trauma
12. Checklist de prompts para usar amanhã
13. O que não terceirizar para a IA
14. O ganho estratégico para empresas com Delphi
15. Plano de ação de 7 dias
16. Três mensagens para levar para casa
17. Apêndices: prompts prontos e mini glossário

1. Por que esta conversa importa

Há mais de 20 anos dizem que o Delphi morreu. Enquanto isso, sistemas Delphi continuam emitindo nota, fechando folha, controlando estoque, processando pedidos, integrando sistemas e sustentando empresas. O obituário foi escrito muitas vezes. Os sistemas, aparentemente, não receberam o memorando.

Esta não é uma defesa nostálgica do passado. Nostalgia é boa para música, foto antiga e cheiro de manual impresso. Para software, o que interessa é valor, risco, continuidade, governança e capacidade de evolução.

A pergunta madura não é: Delphi está vivo ou morto? A pergunta correta é: quanto conhecimento de negócio ainda está codificado em sistemas Delphi críticos, e como esse conhecimento pode ser preservado, compreendido e transformado em arquitetura evolutiva?

Mensagem para levar

O legado não é necessariamente o problema. O problema é conhecimento valioso ficar preso em estruturas difíceis de entender, testar, integrar e evoluir.

2. O que realmente envelhece em um sistema

Nem todo sistema antigo é ruim. Um sistema antigo pode ser estável, valioso e profundamente aderente ao negócio. O que envelhece mal não é apenas a data do código. O que envelhece mal é a perda de clareza.

Dimensão	Quando é patrimônio	Quando vira risco
Código que funciona	Processa regras importantes, sustenta operações e entrega valor todos os dias.	Ninguém entende exatamente por que funciona, e cada alteração vira medo.
Conhecimento de negócio	Representa anos de aprendizado operacional da empresa.	Fica escondido em formulários, eventos, procedures e validações espalhadas.
Equipe	Conhece o histórico, as decisões e os atalhos que mantiveram o sistema vivo.	Depende de poucas pessoas e sofre com documentação frágil.
Tecnologia	Continua produtiva para o que precisa entregar.	Dificulta web, mobile, APIs, nuvem, segurança e integração moderna.

Em muitos sistemas Delphi, a interface foi, por anos, o ponto natural de concentração das regras. Botões, grids, datasets, eventos e validações cresceram juntos. Isso não aconteceu por incompetência. Aconteceu porque o sistema precisava funcionar, o cliente precisava faturar e o prazo era ontem.

O problema aparece quando a empresa quer integrar, expor API, criar frontend web, automatizar processos, aplicar IA ou migrar por fatias. Nesse momento, o conhecimento que estava prático vira opaco.

3. A armadilha da reescrita total

Trocar linguagem pode parecer modernização. Muitas vezes, é apenas trocar o tipo de dívida. A promessa sedutora aparece com frases conhecidas: vamos reescrever tudo do zero; agora vai ficar moderno; em seis meses resolvemos.

A realidade costuma ser menos cinematográfica e mais boleto corporativo. Reescritas totais carregam riscos altos: regra perdida, escopo infinito, bug novo, equipe cansada, prazo estourado e um sistema novo repetindo problemas antigos.

Cuidado estratégico

Reescrita total pode ser necessária em alguns casos, mas não deve ser o reflexo automático. Antes de trocar a pilha tecnológica, é preciso entender o comportamento real que sustenta o negócio.

Modernizar não é apagar. Modernizar é transformar conhecimento em arquitetura evolutiva. Isso exige método, documentação, testes, separação de responsabilidades e comunicação honesta com o negócio.

4. A tese do Delphi Aumentado por IA

IA não é atalho mágico. É amplificador de engenharia quando existe método.

Tese central

A modernização de sistemas Delphi deve começar pela preservação do conhecimento de negócio, pela melhoria da arquitetura e pelo uso inteligente da IA.

O objetivo não é trocar Delphi por moda. O objetivo é reduzir risco, aumentar clareza e abrir caminhos para web, APIs, nuvem, integração e novos produtos.

Delphi Aumentado por IA é uma forma de trabalhar. Junta três elementos:

- **Delphi:** código existente, componentes, domínio de negócio, experiência da equipe e ecossistema Embarcadero.
- **IA:** leitura assistida, explicação de código, geração de hipóteses, documentação, testes e refatoração.
- **Engenharia:** critério técnico, revisão humana, arquitetura, segurança e evolução controlada.

Frase de bolso

IA sem engenharia vira chute com autocomplete. Engenharia com IA vira vantagem operacional.

5. Onde a IA ajuda de verdade

O valor da IA está em acelerar tarefas cognitivas que consomem muito tempo. Ela não substitui experiência, mas pode reduzir a distância entre o código existente e o entendimento compartilhado pela equipe.

18. Explicar units, forms, métodos e dependências em linguagem técnica e de negócio.
19. Localizar regras de negócio escondidas na interface.
20. Gerar documentação técnica e funcional revisável.
21. Sugerir refatorações incrementais e classes de serviço.
22. Criar cenários de teste, casos extremos e critérios de aceite.
23. Apoiar desenho de APIs, DTOs e contratos REST.

Regra de ouro

A IA propõe. O desenvolvedor avalia. A arquitetura decide. O teste confirma. A produção agradece.

O ganho não está em pedir: melhore este código. O ganho está em pedir análise estruturada, critérios, hipóteses, riscos e próximos passos. Prompt bom não é poesia. Prompt bom é engenharia com instrução clara.

6. Modernização inteligente: do medo ao método

A modernização precisa ser incremental, rastreável e segura. Não é big bang. É uma esteira de compreensão, extração, encapsulamento e evolução.

Etapa	Objetivo	Resultado esperado
Legado	Selecionar módulo VCL/FMX, forms e units relevantes.	Escopo pequeno, valioso e compreensível.
IA lê	Pedir explicação, resumo e mapeamento do código.	Hipóteses de regras e dependências.
Regras	Separar negócio, validação e cálculo.	Conhecimento explícito e validável.
Serviços	Extrair classes, domínio e casos de uso.	Regra menos acoplada à interface.
APIs	Definir REST, JSON, DTOs e contratos.	Camada de integração clara.
Web	Criar nova interface por fatias.	Evolução com TMS WEB Core, PWA ou dashboards.

O legado continua entregando valor enquanto novas camadas nascem ao redor dele. Esse ponto é decisivo. Modernização responsável não coloca a empresa em coma induzido para depois descobrir se a cirurgia deu certo.

7. Demo visual: o caso do Pedido legado

A demonstração da palestra usa um sistema VCL de pedidos como exemplo. A ideia é mostrar o processo mental, não depender de internet ao vivo, chave de IA, banco de dados real, bug surpresa ou do generoso humor dos deuses do demo day.

7.1 O cenário

- Sistema VCL de pedidos.
- Regra concentrada no botão Confirmar.
- Cálculo de desconto.
- Validação de limite de crédito.
- Baixa de estoque.
- Persistência do pedido.

7.2 O código antes

```
procedure TFrmPedido.btnConfirmarClick(Sender: TObject);
begin
 if Cliente.Bloqueado then
  raise Exception.Create('Cliente bloqueado');

 Total := CalcularTotalPedido;

 if Total > Cliente.LimiteCredito then
  raise Exception.Create('Limite de crédito excedido');

 if Total > 1000 then
  Desconto := Total * 0.05
 else
  Desconto := 0;

 Pedido.Total := Total - Desconto;
 Pedido.Status := 'CONFIRMADO';
 BaixarEstoque(Pedido);
```

```
  SalvarPedido(Pedido);
end;
```

Esse não é um código absurdo. É exatamente o tipo de código comum em sistemas que cresceram durante anos. O problema não é ter existido. O problema é continuar assim quando a empresa precisa evoluir para API, web, integração e testes.

- Evento de botão misturado com regra de negócio.
- Validação de cliente misturada com cálculo comercial.
- Regra de crédito e desconto sem isolamento.
- Persistência e baixa de estoque dentro do fluxo da tela.
- Dificuldade para testar sem abrir a interface.

7.3 Prompt 1: pedir compreensão, não pedir milagre

```
Analise este método Delphi como um engenheiro de software.

1. Explique o que ele faz em linguagem de negócio.
2. Liste as regras de negócio encontradas.
3. Separe responsabilidades de interface, domínio e persistência.
4. Aponte riscos de manutenção.
5. Sugira uma refatoração incremental, sem reescrever todo o sistema.
```

A diferença está na intenção. Não se pede para a IA simplesmente melhorar o código. Primeiro, pede-se compreensão estruturada. IA boa não substitui requisito. Ela ajuda a encontrá-lo no matagal.

7.4 Saída esperada: regras descobertas

Área	Regras identificadas
Cliente	Pedido não pode ser confirmado quando o cliente está bloqueado. Pedido acima do limite de crédito deve ser recusado.
Pedido	O total é calculado a partir dos itens. Pedidos confirmados recebem status CONFIRMADO. Pedidos acima de 1000 recebem 5% de desconto.
Operação	A confirmação do pedido dispara baixa de estoque. A persistência acontece no final do fluxo. Falhas precisam ser tratadas antes de gravar.

Isso é documentação funcional emergindo do código. Depois, alguém do negócio valida, corrige, complementa e transforma esse material em política de negócio, critério de aceite e teste.

7.5 Prompt 2: refatorar por fatia segura

```
Com base nas regras identificadas, proponha uma refatoração incremental.

Restrições:
- Manter o formulário VCL funcionando.
- Criar uma classe TPedidoService.
- Separar validações e cálculo do evento do botão.
- Não alterar banco de dados agora.
- Sugerir testes unitários para os cenários principais.
- Explique o risco de cada mudança.
```

A IA deve trabalhar dentro de limites. Sem restrições, ela pode sugerir uma reforma inteira quando você precisava trocar uma tomada. Bonito no slide, perigoso na sexta-feira.

7.6 O código depois: serviço de domínio como primeira ponte

```
type
  TPedidoService = class
```

```
 public
   procedure ConfirmarPedido(APedido: TPedido; ACliente: TCliente);
 end;

procedure TPedidoService.ConfirmarPedido(APedido: TPedido; ACliente: TCliente);
begin
 ValidarCliente(ACliente);
 ValidarLimiteCredito(APedido, ACliente);
 AplicarPoliticaDeDesconto(APedido);
 APedido.Status := 'CONFIRMADO';
end;

procedure TFrmPedido.btnConfirmarClick(Sender: TObject);
begin
 PedidoService.ConfirmarPedido(Pedido, Cliente);
 BaixarEstoque(Pedido);
 SalvarPedido(Pedido);
end;
```

A refatoração não cria arquitetura perfeita de primeira. E tudo bem. O ganho inicial está em retirar regra do evento de botão, melhorar legibilidade, tornar parte do comportamento testável, reduzir acoplamento e abrir caminho para API.

7.7 Prompt 3: transformar regra em contrato de API

```
A partir da classe TPedidoService, proponha um contrato REST para confirmar pedido.

Entregue:
1. Endpoint sugerido.
2. JSON de entrada.
3. JSON de saída.
4. Códigos HTTP possíveis.
5. Regras de validação no backend.
6. Observações de segurança.

POST /api/pedidos/{id}/confirmacao

Entrada:
{
 "clienteId": 123,
 "pedidoId": 987
}

Saída:
{
 "pedidoId": 987,
 "status": "CONFIRMADO",
 "total": 1425.50,
 "desconto": 71.28
}
```

A API nasce melhor quando a regra de negócio está clara. Não é primeiro REST, depois entendimento. É primeiro compreensão, depois contrato.

8. Onde entra o TMS WEB Core

O TMS WEB Core entra como ponte para aplicações web modernas mantendo a produtividade mental do Delphi. Ele permite pensar em frontend web, componentes, eventos e propriedades, mas o alvo deixa de ser o desktop e passa a ser o navegador.

- **Frontend web:** aplicações SPA, Pascal compilado para JavaScript, HTML/CSS/JS no navegador e publicação em servidor web.
- **Experiência Delphi:** formulários, componentes, eventos, propriedades e produtividade visual.
- **Cuidado essencial:** WEB Core não é VCL no navegador. Acesso a dados deve passar por API. Regra sensível fica no backend.

Pergunta certa

A pergunta não é: como levo meu form para o browser? A pergunta é: como exponho valor de negócio de forma segura e evolutiva?

9. Arquitetura recomendada com TMS

Uma arquitetura saudável separa o frontend web, a comunicação HTTP/JSON, a API REST e o domínio de negócio. A TMS oferece peças coerentes para esse caminho, mas a arquitetura continua sendo responsabilidade do desenvolvedor.

Camada	Papel	Cuidados
TMS WEB Core	SPA no navegador, frontend web, PWA opcional, interface moderna.	Não colocar regra sensível nem credenciais no cliente.
HTTP/JSON	Contratos, DTOs, respostas claras e erros compreensíveis.	Padronizar status, mensagens, versionamento e validações.
TMS XData	REST API e serviços Delphi.	Proteger endpoints, autenticar, autorizar e registrar erros.
Domínio + Banco	Regras, persistência, FireDAC/Aurelius quando aplicável.	Separar negócio de infraestrutura sempre que possível.

O benefício estratégico é mostrar que o desenvolvedor Delphi pode participar do mundo web sem abandonar sua base técnica. Mas precisa respeitar as regras da web: browser é cliente, HTTP é a fronteira, API é contrato e segurança não é enfeite.

10. O que muda quando o alvo é web

No desktop	Na web
Estado na aplicação.	Browser como cliente, com estado mais explícito e controlado.
Acesso local ou direto a recursos.	Acesso mediado por APIs e permissões.
Form + dataset como centro da experiência.	Frontend, serviços, DTOs, contratos e backend.
Controle maior do ambiente.	Múltiplos navegadores, rede, latência e segurança.
Deploy por executável.	Deploy de frontend, backend, configuração, CORS, HTTPS e monitoramento.

A linguagem ajuda. A arquitetura manda. Com IA, a equipe pode mapear diferenças, gerar checklists, criar planos de migração, sugerir testes e revisar riscos. Mas a decisão final precisa ser técnica, consciente e verificável.

11. Roteiro prático para modernizar sem trauma

Este roteiro transforma a ideia em execução. Ele pode ser usado como checklist inicial para qualquer módulo legado que precise evoluir.

24. Escolha um módulo pequeno, valioso e compreensível.

25. Use IA para explicar código e mapear regras.
26. Valide as regras com alguém do negócio.
27. Extraia serviços sem quebrar a tela existente.
28. Crie testes para proteger comportamento atual.
29. Exponha API com contrato claro.
30. Construa uma nova interface web por fatia.

Critério de sucesso

A versão nova precisa preservar o comportamento essencial, reduzir risco e abrir espaço para evolução. Não basta parecer moderna. Precisa ser mais governável.

12. Checklist de prompts para usar amanhã

Prompts não substituem competência. Eles canalizam competência. Em vez de pedir tudo de uma vez, separe a conversa com a IA por intenção.

Intenção	Prompt base
Análise	Explique este código em linguagem de negócio. Liste regras, exceções, dependências e riscos. Separe interface, domínio, dados e integração.
Refatoração	Sugira uma refatoração incremental. Preserve comportamento atual. Indique riscos, testes e ordem de implementação.
Modernização	Proponha contrato REST. Desenhe DTOs. Liste validações no backend. Sugira estratégia para frontend web.
Validação	Revise esta solução como arquiteto. Aponte riscos, premissas frágeis, acoplamentos e cenários de falha.
Testes	Crie cenários de teste para comportamento atual, casos extremos, falhas de validação e regressões prováveis.

O segredo é pedir que a IA trabalhe como assistente de engenharia, não como oráculo. Oráculo em software costuma cobrar caro e entregar sprint atrasada.

13. O que não terceirizar para a IA

A IA é útil, mas não é responsável técnica pela produção. Em projetos reais, a responsabilidade continua com pessoas, processos e decisões verificáveis.

Não terceirize	Sempre revise	Sempre proteja
Decisão arquitetural, segurança, privacidade, critério de aceite e responsabilidade legal.	Código gerado, dependências, tratamento de erro, performance e compatibilidade.	Dados sensíveis, credenciais, segredos, código proprietário e contexto estratégico.

Governança

IA aumenta capacidade. Não transfere responsabilidade. Quanto mais crítico o sistema, mais importante é explicitar critérios, revisar saídas e proteger dados.

14. O ganho estratégico para empresas com Delphi

A vantagem está em transformar legado em plataforma de evolução. Para gestores, isso significa risco menor, velocidade maior e mais opções futuras. Para desenvolvedores, significa clareza, produtividade e relevância profissional.

- **Menos risco:** compreensão antes da mudança, testes antes da migração e fatia antes do big bang.
- **Mais velocidade:** IA acelera leitura, documentação viva e refatoração orientada.
- **Mais futuro:** APIs, web, nuvem e IA aplicada ao negócio.
- **Mais valor:** conhecimento preservado, equipe empoderada e negócio protegido.

O ponto mais importante é que a modernização deixa de ser uma guerra entre gerações tecnológicas. Ela vira uma ponte: o que funcionou no passado é compreendido, protegido e transformado para sustentar o futuro.

15. Plano de ação de 7 dias

Para quem quer começar sem depender de projeto gigantesco, este plano transforma a palestra em ação prática.

Dia	Ação
1	Escolha uma tela ou fluxo pequeno, mas relevante. Exemplo: confirmar pedido, calcular comissão, liberar crédito ou fechar caixa.
2	Cole o método principal em uma ferramenta de IA segura e peça explicação em linguagem de negócio.
3	Liste regras, exceções, dependências e riscos. Separe o que é interface, domínio, dados e integração.
4	Valide as regras com alguém que conhece o processo real. Corrija nomes, exceções e políticas.
5	Proponha uma primeira classe de serviço ou caso de uso, preservando a tela atual.
6	Crie cenários de teste para proteger o comportamento existente.
7	Desenhe um contrato REST preliminar e avalie se o fluxo pode ganhar uma nova interface web por fatia.

Comece pequeno

O primeiro objetivo não é modernizar o sistema inteiro. É provar que a equipe consegue compreender, documentar, proteger e evoluir uma fatia real do legado.

16. Três mensagens para levar para casa

1. Legado não é lixo

Sistemas antigos podem carregar décadas de conhecimento operacional. Antes de descartar, entenda.

2. IA não é mágica

Ela acelera análise, documentação e refatoração. Mas precisa de método, contexto e revisão.

3. Arquitetura é ponte

APIs, serviços e web devem nascer da regra de negócio compreendida, não do hype.

Mensagem final

Não modernize apagando a memória da empresa. Modernize transformando essa memória em vantagem competitiva.

Apêndice A - Prompts prontos para copiar e adaptar

Prompt de compreensão

```
Analise este código Delphi como um engenheiro de software.
Explique o que ele faz em linguagem de negócio.
Liste regras, validações, exceções, dependências e riscos.
Separe responsabilidades de interface, domínio, dados e integração.
Não reescreva ainda. Primeiro, apenas explique e organize.
```

Prompt de refatoração incremental

```
Com base nas regras identificadas, proponha uma refatoração incremental.
Preserve o comportamento atual.
Mantenha a tela funcionando.
Extraia apenas uma responsabilidade por vez.
Indique ordem de implementação, riscos e testes necessários.
```

Prompt de contrato REST

```
A partir desta regra de negócio, proponha um contrato REST.
Inclua endpoint, método HTTP, JSON de entrada, JSON de saída, códigos HTTP, validações no backend e observações de segurança.
Explique também o que não deve ficar no frontend.
```

Prompt de revisão arquitetural

```
Revise esta proposta como arquiteto de software.
Aponte acoplamentos, riscos de segurança, riscos de manutenção, premissas frágeis e dependências ocultas.
Sugira uma alternativa mais simples, incremental e segura.
```

Prompt de testes

```
Crie cenários de teste para este fluxo.
Inclua comportamento esperado, casos extremos, entradas inválidas, falhas de integração, regras de negócio e riscos de regressão.
Organize os testes em linguagem compreensível para desenvolvedores e pessoas do negócio.
```

Apêndice B - Mini glossário

Termo	Sentido prático neste material
Legado	Sistema existente que continua carregando valor, regras e dependências importantes.
Refatoração	Melhoria interna do código preservando comportamento observável.
Serviço de domínio	Classe ou componente que concentra regras de negócio fora da interface.
API REST	Contrato HTTP para expor operações e dados de forma padronizada.

DTO	Objeto de transporte de dados usado entre camadas ou sistemas.
TMS WEB Core	Caminho para frontend web com produtividade mental próxima ao Delphi.
TMS XData	Caminho para criar APIs REST/JSON e serviços Delphi no backend.
PWA	Aplicação web progressiva, instalável e com experiência mais próxima de app.

Sobre este material

Este e-book acompanha a palestra Delphi Aumentado por IA: Modernização Inteligente de Sistemas Legados sem Perder o Conhecimento do Negócio, apresentada por Prof. Marcelo Nogueira na Embarcadero Conference 2026.

Use este material como ponto de partida para estudar, ensinar, conversar com equipes e iniciar um plano incremental de modernização de sistemas Delphi com apoio de IA, método e responsabilidade técnica.

Acesso aos materiais

Link: https://bit.ly/econ_mn_2026

Materiais da palestra

https://bit.ly/econ_mn_2026

CTA
TECNOLOGIA
EDUCACIONAL